# AIR-GB Lightning: Native Multi-Path RGB Routing & Local QVAC AI

Solving Single-Path RGB Liquidity Stranding & High-Fee Detours on the Bitcoin Lightning Network



THE CORE BREAKTHROUGH FOR RGB SMART CONTRACTS

## First Native Multi-Path Payment (MPP) Engine for RGB Assets on Lightning,

## Guided by Local QVAC AI (Qwen3.5-0.8B + TurboVec RAG) & 4 Deterministic Guardrails

Validated head-to-head against stock single-path rgb-lightning-node across 20 sequential scenarios on a 20-node live regtest topology.

OVERALL BENCHMARK SUCCESS

**35% → 65%**

13 / 20 vs. 7 / 20 Scenarios

81.25% on Reachable Scenarios (13 / 16)

+85.7% more completed transfers

MATCHED-PAIR ROUTING FEES

**-78.95%**

1,200 msat vs. 5,700 msat

Case #11: 0 msat (1 hop) vs. 5,100 msat

Eliminates channel-drain fee detours

MULTI-PATH & RETURN FLOWS

**3 / 3 MPP**

60+40 RGB & 40+40 RGB Splits

190 RGB Returned to Alice (vs. 0)

Fixed LDK multi-HTLC color_commitment

ON-NODE RUST PLANNER SPEED

**0.66-1.35 ms**

4 Native Rust Graph Solvers

100% Local AI + 0 Hallucination Risk

52 / 52 safety contract tests passing

### 1. Custom Rust RLN + LDK MPP Engine

`yen` (K-shortest), and `ldk_dijkstra`.

- Proportional BTC carrier splitting (>= 3,000 sat/shard) + atomic dual-asset aggregation at recipient.
- Solved upstream LDK multi-HTLC commitment coloring bug via exact claimed-preimage RGB delta accounting.

`Commits: rgb-lightning-node@7106edd • rust-lightning@2f19847`

### 2. Local QVAC AI + Node-Scoped TurboVec RAG

- `EMBEDDINGGEMMA_300M_Q4_0`: 768-dim embeddings in QVAC `turbovec-q4` SIMD index + HyperDB reranking.
- `QWEN3_5_0_8B_MULTIMODAL_Q4_K_M`: 3 structured JSON passes (NL intent compiler, channel scorer, algo selector).

+ request-scoped RAM wipe (`EphemeralRoutingContext`).

`100% Local Loopback (127.0.0.1:11434) • Zero Cloud APIs`

### 3. Deterministic Safety & Live Verification

- 4 Deterministic Guardrail Gates block invented IDs, prompt injection, and liquidity hallucinations.
- Cryptographic `SHA-256` policy binding + `secp256k1` point validation + bilateral balance overrides.
- Reproducible 20-node Docker regtest benchmark starting from identical state hash `fc59bc78...8b45`.

`Presentation Structure: 8 Technical Slides + Live Demo`

PROBLEM STATEMENT

# The Problem: Why Single-Path RGB Lightning Breaks at Scale

Stock rgb-lightning-node routes along a single path (max_path_count = 1), stranding split RGB liquidity and inflating fees

TIMING SLOT 0:30 - 1:10



### 1. Split-Corridor Liquidity Stranding

- Stock RLN hardcodes with_max_path_count(1) inside rust-lightning RouteParameters.
- When Alice has 60 RGB on Corridor A (Bob) and 40 RGB on Corridor B (Carol), any transfer > 60 RGB fails immediately.

✓ AIR-GB unlocks 60+40 and 40+40 RGB splits.

### 2. Leaf-Node Trapping & 5,100 msat Detours

- Single-path transfers drain primary channels ***in one direction (Case 07 drains Alice→Bob).***
- Return transfers fail due to LDK 1% channel reserve (10M msat) + single-path limits.
- ***Case 11 (1 RGB Alice→Bob) is forced into a*** 3-hop detour costing 5,100 msat vs. 0 msat!

### 3. Intent & Historical Failure Blindness

- Standard LDK Dijkstra ignores user fee caps, node exclusions, and probing privacy rules.
- No RAG memory of RGB contract-specific TemporaryChannelFailure or HTLCTimeout.

✓ AIR-GB adds local QVAC RAG + 4 Rust planners.

## Why Stock Single-Path RGB Lightning Breaks at Scale

Three compounding routing failures observed in standard rgb-lightning-node (max_path_count = 1) across our 20-node regtest benchmark

### 1. Split-Corridor Liquidity Stranding

Stock RLN hardcodes `with_max_path_count(1)`. Parallel RGB corridors cannot be combined:



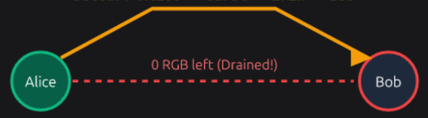

Benchmark Impact (Cases 02, 03, 04):

- Case 02 (100 RGB) & Case 04 (80 RGB) fail immediately with FailedToFindRoute.
- Upstream RLN color_commitment() also crashes if >1 RGB HTLC is active on the same channel due to stale last_rgb_payment_info file state.

### 2. Leaf Stranding & 5,100 msat Detours

One-way single-path drains primary channels and traps liquidity at leaf nodes (Judy):

***Step A: Case 07 drains Alice → Bob (60 → 0 RGB)***
Step B: Case 08 return fails (1% BTC reserve)
***Detour: Alice → Carol → Erin → Bob***




Case 11 (1 RGB): 5,100 msat fee (3 hops) vs 0 msat!

Benchmark Impact (Cases 08, 10, 11, 12, 13, 15):

- 0 / 3 return flows succeed in Baseline (0 RGB returned to Alice vs 190 RGB in AIR-GB).
- ***Mid-network hops (Bob→Erin, Carol→Frank, Frank→Ivan) all fail because upstream channels*** were completely emptied by single-path routing.

### 3. Intent & Telemetry Blindness

Standard LDK Dijkstra uses static edge weights with zero awareness of user intent or RGB history:

What Stock LDK Dijkstra Ignores:

- Natural-Language Fee Caps: Cannot parse "Cap fees at 500 msat, avoid Bob"
- Privacy vs. Probing Trade-offs: No per-payment toggle between MPP & single-path
- First-Party RGB Failure Memory: No RAG index of past RGB HTLC timeouts, congestion ratios, or per-algorithm success rates

How AIR-GB Solves All Three:

- ✓ Native 4-Layer Rust RGB MPP (MinCostFlow)
- ✓ Multi-Objective Pareto & Yen K-Shortest
- ✓ QVAC Local LLM + TurboVec RAG Telemetry
- ✓ 4-Gate Deterministic Physical Guardrails

Net Result on the 20-Scenario Live Regtest Benchmark (topology-mpp.json):

Stock Baseline: 7 / 20 Succeeded (35.0%) • 0 MPP Splits • 5,700 msat Matched Fees
Strands 100% of RGB at leaf nodes; fails all large transfers & return flows

AIR-GB (AI + MPP): 13 / 20 Succeeded (65.0% | 81.3% Reachable) • 1,200 msat Fees
3 / 3 MPP Splits • 190 RGB Returned to Alice • -78.95% Routing Fee Reduction

# End-to-End AIR-GB Routing Architecture Pipeline

How natural-language user policies and node-scoped RAG telemetry drive our 4 native Rust graph planners



## Stages 1 & 2: NL Policy + TurboVec RAG

- ***Stage 1 (interpret.ts → compile.ts):***
  Compiles fee caps, node exclusions & privacy into a SHA-256 fingerprinted CompiledPolicy.
- Stage 2 (channel-scoring.ts):
  Queries node-scoped QVAC TurboVec RAG and scores live channels + Pareto weights.

## Stages 3 & 4: Strategy Selector + Rust MPP

- Stage 3 (routing-selector.ts):
  Evaluates single-path vs. MPP feasibility in ephemeral RAM (wiped in finally).
- Stage 4 (routes.rs & channel.rs):
  Executes min_cost_flow, multi_objective, yen, or ldk_dijkstra in 0.66-1.35 ms.

## Dynamic Algorithm Selection in Benchmark

- Across all 20 benchmark scenarios, AIR-GB never used static ldk_dijkstra:
- ✓ min_cost_flow (max_paths: 2) for MPP splits (Cases 02, 03, 04: 100 RGB & 80 RGB).
- ✓ multi_objective (max_paths: 1) for single-hop RGB conservation & returns (08, 10, 11..15).

## Routing Architecture: Baseline Single-Path vs. AIR-GB AI + MPP Pipeline

How AIR-GB transforms natural language intents and first-party channel telemetry into guardrailed multi-path RGB Lightning settlements

BASELINE STACK (rgb-lighning-stats-no-ai)

**1. Payment Request**
POST /sendpayment
Ignores natural language hints, fee budgets & privacy intents

→

**2. Static Single-Path Config**
with_max_path_count(1)
Fixed LDK ProbabilisticScorer
No RAG memory of RGB channel failures

→

**3. LDK Standard Dijkstra**
Single-Corridor Search Only
Cannot aggregate 60 + 40 RGB
Strands liquidity at leaf nodes

→

**Baseline Result**
7 / 20 Succeeded (35%)
0 MPP Splits • 5,700 msat fees
5,100 msat 3-hop detour on #11

AIR-GB AI + MPP STACK (air-gb-lightning + QVAC Local LLM)

**Stage 1: NL Policy Compiler**
***interpret.ts → compile.ts***
- Parses user prompt constraints:
  - Fee limits (e.g., max_fee_msat)
  - Node exclusions (excluded_nodes)
  - Privacy: avoid_extra_probing
  - Priority: low_fee / reliability

→

**Stage 2: QVAC RAG + Scorer**
channel-scoring.ts (TurboVec)
- Node-scoped 1st-party SQLite + RAG
- Qwen3.5-0.8B classifies live channels
- Physical Liquidity Guardrail:
  Verified local BTC/RGB balances cannot be downgraded to insufficient_liquidity

→

**Stage 3: Strategy Selector**
routing-selector.ts
- EphemeralRoutingContext (RAM wipe)
- Filters 0-RGB channels from MPP count
- Deterministic Guardrails:
  - Upgrade to min_cost_flow if >1 path
  - Cap max_paths=2; force 1 on privacy

→

**Stage 4: Rust LDK MPP**
routes.rs + channel.rs
- 4 Native Graph Planners:
  1. min_cost_flow (RGB MPP)
  2. multi_objective (Pareto)
  3. yen (K-shortest paths)
  4. ldk_dijkstra (Fallback)

**Critical Protocol & Infrastructure Fixes Engineered Under the Hood**

1. Multi-HTLC RGB Commitment Coloring Fix (color_commitment in rgb_utils/mod.rs & ln/channel.rs):
   Stock RLN subtracted HTLC RGB amounts from stale last_rgb_payment_info. Fixed to compute exact settled delta (value_to_self_claimed_rgb - value_to_remote_claimed_rgb) and enforce available_outbound_rgb / available_inbound_rgb.
2. Gossip Convergence & Peer Reconnect Readiness (assertPaymentReady + /benchmark/gossip-status):
   Eliminates cold-start routing races by verifying all 15 channels have bidirectional channel_update gossip before Case 01.

**AIR-GB Benchmark Result**
13 / 20 Succeeded (65%)
81.25% on Reachable Scenarios (13/16)
1,200 msat Matched Fees (-78.95%)
0.66–1.35 ms Rust Planner Execution

# Local QVAC AI Models & Privacy-First TurboVec RAG

Two specialized local models running on loopback (127.0.0.1:11434) with strict first-party Lightning privacy



### Model 1: EMBEDDINGGEMMA_300M_Q4_0

- 300M params, Q4_0, 768-dim mean pooling.
- Step 0 (Post-Payment Ingest): Embeds <150-token first-party payment episodes into TurboVec (turbovec-q4 SIMD + HyperDB reranking).
- Step 2a (Pre-Payment Search): Retrieves top-4 relevant episodes via /v1/vector_stores/search.

### Model 2: QWEN3_5_0_8B_MULTIMODAL_Q4_K_M

- Qwen 3.5 0.8B (Q4_K_M, temp: 0, strict JSON).
- Step 1: Extracts NL routing constraints.
- Step 2b: Classifies live channels (optimal, good, congested, avoid) & Pareto weights.
- Step 3: Proposes routing algorithm & MPP split count from feasibility + fail rates.

### Privacy Rule: First-Party Only + RAM Wipe

- assertFirstPartyEpisode() rejects any remote balances, 3rd-party route hops, or invoices.
- EphemeralRoutingContext.wipe() zeroes RAM capacity bounds immediately in finally {}.

✓ Zero external cloud calls; AI holds no keys.

## Local QVAC AI Stack: What Models We Use & What Each Model Does

100% Local Inference via qvac serve --openai (127.0.0.1:11434) • Zero External Cloud Calls • Strict JSON-Schema Grammar

### Model 1: EMBEDDINGGEMMA_300M_Q4_0 (alias: my-embed)

`300M params • 4-bit Q4_0 • 768-dim vectors • Mean Pooling • GPU`

#### STEP 0: Post-Payment Telemetry Ingestion

Triggered after every settled/failed payment or fixture seed:

- Formats a <150-token single-paragraph episode (`formatTelemetryDocument`)
- ***Uploads via POST /v1/files → POST /v1/vector_stores/:id/files***
- Why ragTurbovec: true? Uses native SIMD `turbovec-q4` + HyperDB full-precision reranking so incremental ingests never suffer 50-chunk FIFO eviction.

#### STEP 2a: Pre-Payment Semantic Retrieval

Runs inside `scoreChannelsWithQvac()` before every payment:

- Embeds semantic query: asset ID (RGB vs BTC), `amountMsat`, peer aliases
- Queries `POST /v1/vector_stores/:id/search` (`max_num_results: 4`)
- Returns top-4 `RagTelemetryHit[]` (past HTLC timeouts, fee spikes, or high-outbound RGB successes on the node's own channels) to feed Model 2.

✓ Includes deterministic SQLite keyword fallback if vector store is cold.

### Model 2: QWEN3_5_0_8B_MULTIMODAL_Q4_K_M (qwen-fast / my-llm)

`Qwen 3.5 0.8B • Q4_K_M GGUF • ctx: 8192 • temp: 0 • Strict JSON Schema`

#### STEP 1: NL Constraint Extraction (interpret.ts)

- ***Endpoint: POST /api/constraints/interpret → /v1/chat/completions***
- Schema: `extracted.schema.json` (feeLimit, excludedNodeReferences, priority: lowest_fee | reliability | balanced, privacyRequest: avoid_extra_probing)

***→ Compiled by compile.ts into SHA-256 fingerprinted CompiledPolicy***

#### STEP 2b: Channel Classifier & Weight Tuner (channel-scoring.ts)

- Input: Live `/listchannels` ratios + Top-4 TurboVec RAG episodes

#### STEP 3: Algorithm & MPP Split Selector (routing-selector.ts)

- Input: Scored channels + algorithm fail rates + EphemeralRoutingContext

`multi_objective` (Pareto), `yen` (K-shortest), or `ldk_dijkstra` + `max_paths (1..8)`

### Lightning Privacy Invariant: "Don't Hold Other People's Data" (First-Party Scope + Ephemeral RAM Wipe)

- `assertFirstPartyEpisode()`: Rejects any object containing third-party route hops, remote node balances, or raw invoices with PRIVACY_VIOLATION.

SAFETY & VERIFICATION

# Zero-Hallucination Design: 4 Deterministic Guardrail Gates

Why a fast 0.8B LLM can never hallucinate a channel ID, violate a fee cap, or misclassify physical liquidity

TIMING SLOT 2:30 - 3:10



## Gates 1 & 2: Grammar & Crypto Binding

- Gate 1 (dynamicInferenceSchema): Locks channelId to live /listchannels enum and strips 66-char hex pubkeys during generation.
- Gate 2 (parseExtraction + compile.ts): Verifies every fee/alias is a literal prompt substring & binds SHA-256 paymentFingerprint.

## Gate 3: Bilateral Physical Liquidity Override

- Computes deterministic fallback from live BigInt BTC and RGB balances first.
- Blocks False Positives: Offline/underfunded channels cannot be promoted to optimal/good.
- Blocks False Negatives: Funded MPP channels cannot be downgraded to insufficient_liquidity.

## Gate 4: Authoritative MPP & Rust Checks

- Excludes 0-RGB channels from RGB MPP counts.
- Forces min_cost_flow (k >= 2) whenever !singlePathPossible && mppPossible.
- Forces max_paths = 1 on avoid_extra_probing.

✓ Payee pubkey is never added to excluded_nodes.

## Zero-Hallucination Architecture: 4 Deterministic Guardrail Gates

How AIR-GB guarantees no 0.8B LLM hallucination, truncation, or liquidity mistake can ever reach the Lightning network

### GATE 1: Grammar Locking

`dynamicInferenceSchema`

**Threat Blocked:**

Small 0.8B model inventing channel IDs, corrupting 66-char hex pubkeys, or looping array items until finish_reason === "length".

**Deterministic Enforcement:**

- Per-request JSON Schema built from live RLN /listchannels
- `channelId: { enum: liveIds }`
- `minItems = maxItems = N`
- 66-char `peerNodeId` stripped from LLM prompt & re-attached deterministically in post-pass
- Any invented channel ID is dropped during array mapping

### GATE 2: Semantic & Crypto

`parseExtraction + compile.ts`

**Threat Blocked:**

Prompt injection ("send now"), hallucinated fee caps, inverted negations ("don't exclude Bob"), or excluding the invoice payee.

**Deterministic Enforcement:**

- Verifies every extracted fee, unit & alias is a literal substring of user text (AI_SEMANTIC)
- Validates 33-byte secp256k1 point via `crypto.ECDH.convertKey()`
- Rejects excluding `payeePubkey` with CONSTRAINT_CONFLICT
- Binds policy to `SHA-256` hash of `{invoice, amount, assetId}`

### GATE 3: Bilateral Liquidity

`sanitizeAndValidateChannelScore`

**Threat Blocked:**

1) Promoting an offline/underfunded channel to optimal/good, OR
2) Falsely downgrading a valid partial MPP channel to avoid.

**Deterministic Enforcement:**

- Computes `fallback` from live BigInt BTC & RGB balances first
- ***If `!isUsable` → forced `avoid`***
- ***If underfunded → forced*** `insufficient_liquidity`
- ***If physically sufficient → LLM*** is blocked from outputting insufficient_liquidity or avoid
- Clamps weights to `[1..1000]`

### GATE 4: Strategy & Rust

`routing-selector.ts + routes.rs`

**Threat Blocked:**

Picking single-path when MPP split is physically required, violating privacy single-path, or counting 0-RGB channels as MPP shards.

**Deterministic Enforcement:**

- `mppContributors` requires both BTC > 0 and RGB > 0
- Rule 1: `avoid_extra_probing` forces `max_paths = 1`
- Rule 2: `!singlePath && mpp`
- Rust `routes.rs` enforces `available_outbound_rgb()` & `max_fee_msat` before wire HTLC

### Full Auditability in Benchmark & Production (selectionSource = "qvac_with_guardrails")

- Every payment decision records both `decision.modelDecision` (raw QVAC proposal) and `decision.recommendedRouting` (post-guardrail policy).

✓ Result: Sub-second local AI adaptability with 100% deterministic financial and cryptographic safety.

Verification Suite: 52 / 52 Fixture & Guardrail Contract Tests Passing (wallet/tests/fixture-scoring.test.ts)

# Under the Hood: 4-Layer Native RGB Multi-Path Payments in Rust

Coupled dual-asset flow decomposition, atomic recipient shard aggregation, and multi-HTLC commitment coloring fix



### Layers 1 & 2: MinCostFlow & BTC Carrier

- Successive shortest augmenting paths split
  100 RGB into 40 RGB (100 msat/hop via Carol)
  + 60 RGB (200 msat/hop via Bob) = 900 msat.
- Proportional BTC carrier allocation enforces
  RLN 3,000,000 msat minimum per colored HTLC:
  Shard 1 = 18M msat, Shard 2 = 12M msat.

### Layer 3: Atomic Dual-Asset Aggregation

- Each UpdateAddHTLC carries (ContractId, u64)
  alongside shared payment_hash & secret.
- Recipient InboundPayment holds shards until
  BOTH sum(msat) >= 30M AND sum(rgb) == 100.

✓ Releases PaymentPreimage atomically across
all shards or fails all back cleanly.

### Layer 4: Multi-HTLC color_commitment() Fix

- Stock RLN read a single stale file
  (last_rgb_payment_info), crashing under MPP.
- Fixed by reserving in-flight RGB via
  available_outbound_rgb / available_inbound_rgb

✓ Computes exact settled delta from claimed
preimages inside build_commitment_transaction.

## How Native RGB Multi-Path Payments (MPP) Work Across All 4 Layers

Walkthrough of Scenario 02-rgb-mpp-100: Splitting 100 RGB + 30,000,000 msat BTC Carrier from Alice to Judy (60 RGB + 40 RGB)

### LAYER 1: Pre-Flight MPP Feasibility (routing-selector.ts)

1. EphemeralRoutingContext Inspects Alice's Live Channels:

- alice-bob : 60 RGB | 990M msat spendable ✓ Partial Contributor #1
- alice-carol: 40 RGB | 990M msat spendable ✓ Partial Contributor #2
- alice-dave : 40 RGB | 990M msat spendable ✓ Backup Contributor

2. Dichotomous Feasibility & Guardrail Decision:

- singlePathPossible = false (Max single channel is 60 RGB < 100 RGB required)
- mppPossible = true (60 + 40 = 100 RGB covered in k = 2 paths)
- Physical Liquidity Guardrail prevents 0.8B LLM from marking 60/40 channels as insufficient_liquidity; forces algorithm: "min_cost_flow", max_paths: 2

### LAYER 2: Rust Min-Cost Flow & Proportional Carrier (routes.rs)

1. Successive Shortest Augmenting Paths on Residual Graph:

- Hop 1 out of Alice bounded by real available_outbound_rgb()
- *Augmenting Path 1 (cheapest: 100 msat/hop): pushes 40 RGB via Carol→Frank→Ivan→Jud*
- *Augmenting Path 2 (next: 200 msat/hop): pushes 60 RGB via Bob→Erin→Heidi→Judy*

2. Proportional BTC Carrier Allocation (MIN_RGB_HTLC_MSAT = 3M):

M_i = max(3,000,000 msat, floor(30,000,000 * f_i / 100))

- Shard 1 (60 RGB): 18,000,000 msat BTC + 600 msat fee (3 hops × 200)
- Shard 2 (40 RGB): 12,000,000 msat BTC + 300 msat fee (3 hops × 100)
- Total Route : 100 RGB | 30,000,000 msat | 900 msat total fee

### LAYER 3: Atomic Multi-Part RGB Aggregation at Recipient (LDK)

Dual-Asset Hold Condition on Shared payment_hash + payment_secret:

HTLC Part 1 (from Heidi)
+60 RGB | +18,000,000 msat

HTLC Part 2 (from Ivan)
+40 RGB | +12,000,000 msat

Judy InboundPayment Aggregator
Σ msat = 30M ≥ 30M (OK)
Σ rgb = 100 == 100 (OK)
✓ Release Preimage Atomically!

- Judy holds Part 1 in memory until Part 2 arrives with the same payment_secret.
- Only when both BTC and RGB sums match the invoice does LDK emit Event::PaymentClaimable and reveal the preimage back along both paths.
- If any part times out or fails, all parts roll back atomically (zero partial loss).

### LAYER 4: Multi-HTLC RGB Commitment Coloring Fix (channel.rs / mod.rs)

Why Stock rust-lightning Crashed on Concurrent / Sequential RGB HTLCs:

- Stock color_commitment() subtracted rgb_offered_htlc from a stale disk snapshot (last_rgb_payment_info) and mutated pending balances before preimage fulfillment.

Our 3-Part Fix in rust-lightning:

1. In-Flight Reservation (available_outbound_rgb / available_inbound_rgb):
   Subtracts all pending_outbound_htlcs & holding_cell_htlc_updates from local_rgb_amount.
2. Exact Claimed-Preimage Delta in build_commitment_transaction:
   delta = value_to_self_claimed_rgb - value_to_remote_claimed_rgb
   Passes exact rgb_balances: (u64, u64) directly into color_commitment().
3. Settle-Only Balance Mutation:
   Updates on-disk channel RGB amounts strictly on InboundHTLCRemovalReason::Fulfill.

BENCHMARK TOPOLOGY

# The 20-Node Regtest Benchmark Topology (topology-mpp.json)

15 colored RGB+BTC channels across 10 core routing nodes (Alice-Judy) + 10 isolated negative-control nodes (Karl-Tara)

TIMING SLOT 3:55 - 4:30



## 1. LDK 1% Reserve Cushion (push_msat: 7M)

- Every 1M sat channel enforces a 1% (10M msat) reserve before Judy can send an outbound HTLC.
- ***After Case 02 (18M + 12M msat split), Ivan→Judy*** had only 2M msat above reserve (< 3M floor!).
- ✓ push_msat: 7,000,000 gives Judy 15M & 9M msat spendable BTC, unlocking reverse MPP returns.

## 2. Differentiated Corridors & Cross-Guards

- 40-RGB branch (Carol-Frank-Ivan): 100 msat/hop
- 60-RGB branch (Bob-Erin-Heidi): 200 msat/hop
  → ***100 RGB MPP split costs 900 msat total.***
- 20-RGB cross-links oriented with 5,000 msat ***fee in Judy→Alice direction to prevent >2-path*** reverse fragmentation across default 1k edges.

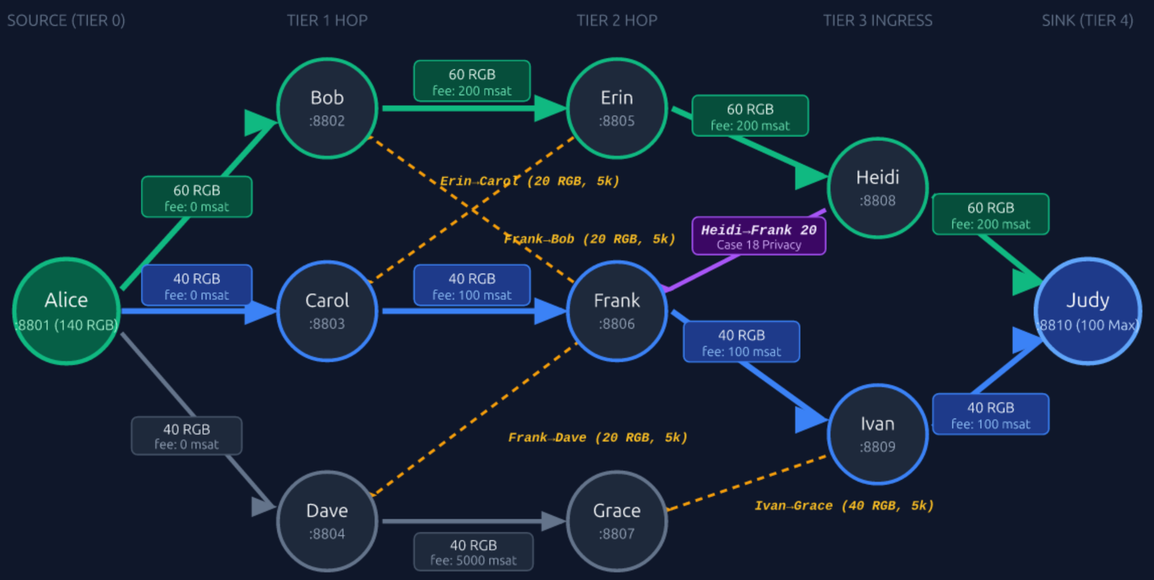
## 3. Privacy Link & Gossip Readiness Gate

- ***Heidi→Frank (20 RGB) enables single-path privacy routing in Case 18 (Heidi→Frank→Ivan).***
- assertPaymentReady + /benchmark/gossip-status verifies all 15 channels have bidirectional gossip before Case 01 begins.

## 20-Node Regtest Benchmark Topology (topology-mpp.json vs topology.json)

15 Colored RGB + BTC Channels across 10 Core Nodes (Alice–Judy) + 10 Isolated Peripheral Nodes (Karl–Tara) • 1,000,000 sat / channel • push_msat: 7,000,000

60 RGB Main Branch (200 msat/hop = 600 msat) · 40 RGB Low-Fee Branch (100 msat/hop = 300 msat) · Cross-Link (5,000 msat fee) · Privacy Link




### Why topology-mpp.json?

4 protocol bottlenecks solved:

**1. LDK 1% Reserve Cushion**
push_msat: 7,000,000 (7k sat). Clears LDK's 10M msat reserve so Judy can route 100 RGB back (Case 03).

**2. Low-Fee MPP Corridors**
Carol-Frank-Ivan: 100 msat/hop (300)
Bob-Erin-Heidi: 200 msat/hop (600)
100 RGB split = 60+40 for 900 msat.

**3. Reverse Cross-Link Guards**
Cross-edges set to 5,000 msat in ***Judy→Alice direction (>1k default).*** Prevents >2 path MCF fragmentation.

***4. Heidi→Frank Privacy Hop***
***Reversed Frank-Heidi to Heidi→Frank*** (20 RGB) so Heidi can reach Ivan in 18-rgb-privacy (single-path).

### 10 Peripheral Nodes (Karl – Tara) • Negative Control Reachability Island (Cases 19 & 20)

Online with funded wallets and P2P connections, but 0 open Lightning channels in the benchmark fixture — verifying that both routers cleanly reject unreachable destinations.

| Karl | Leo | Mia | Nico | Olive | Paul | Quinn | Ruth | Saul | Tara |
|---|---|---|---|---|---|---|---|---|---|
| :8811 • 0 ch | :8812 • 0 ch | :8813 • 0 ch | :8814 • 0 ch | :8815 • 0 ch | :8816 • 0 ch | :8817 • 0 ch | :8818 • 0 ch | :8819 • 0 ch | :8820 • 0 ch |

EMPIRICAL EVALUATION

# Head-to-Head Benchmark Results & The Case #11 Fee Multiplier

Sequential execution on identical starting channel state (fc59bc78...8b45): Stock Baseline (stock-02) vs. AIR-GB (ai-02)

TIMING SLOT 4:30 - 5:05



## 1. +30 pp Success Rate (+85.7% More Payments)

- ***Overall (20 Cases): 35.0% (7/20) → 65.0% (13/20)***
- ***Reachable (16 Cases): 43.8% (7/16) → 81.3% (13/16)***
- All 4 negative controls (Cases 06, 17, 19, 20) properly rejected by both stacks.
- ✓ 3 / 3 MPP splits (02, 03, 04) succeeded in AIR-GB vs. 0 / 3 in Stock Baseline.

## 2. Why Case #11 Cost 5,100 msat vs. 0 msat

- In Baseline, Case 07 drained all 60 RGB on ***Alice→Bob, and Case 08 failed to return it.***
- When Alice sent 1 RGB to Bob in Case 11, she ***was forced into a 3-hop detour (Alice→Carol →Erin→Bob) costing 5,100 msat (10.1s)!***
- ✓ AIR-GB returned 190 RGB to Alice (03, 08, 10) and routed Case 11 directly in 1 hop for 0 fee.

## 3. Sequential State Divergence (07 & 18)

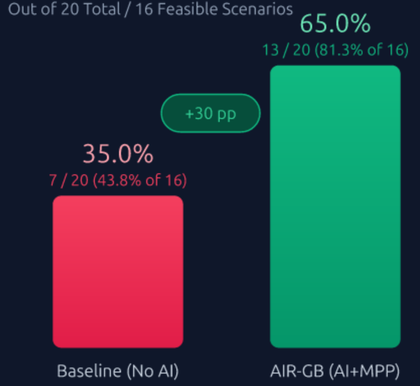
- Cases 07 & 18 only succeeded in Baseline because earlier transfers (02, 04, 15) FAILED, leaving untouched RGB on Alice-Bob & Frank-Ivan.
- In AIR-GB, 04 & 15 succeeded and utilized that liquidity productively earlier in the run.

## Sequential Regtest Benchmark Results: Baseline (No AI) vs. AIR-GB (QVAC + MPP)

Identical Starting State Hash (fc59bc78...8b45) • 20 Live Regtest Scenarios on topology-mpp.json (16 Reachable + 4 Negative Controls)

### 1. Payment Success Rate

Out of 20 Total / 16 Feasible Scenarios




### 2. Routing Fees Paid (Matched Pairs)

Total msat across 5 paired successes (01, 09, 11, 14, 16)




### 3. Multi-Path & Return Flows

Why sequential liquidity stayed balanced in AIR-GB

| | |
|---|---|
| Multi-Path RGB Splits (Cases 02, 03, 04) | |
| Baseline: 0 / 3 | AIR-GB: 3 / 3 (60+40, 40+40) |
| ***Judy → Alice Return Flows (03, 08, 10)*** | |
| Baseline: 0 / 3 | AIR-GB: 3 / 3 (190 RGB returned) |
| Mid-Network Transfers (Cases 12, 13, 15) | |
| Baseline: 0 / 3 (Drained) | AIR-GB: 3 / 3 (0 msat direct) |
| Negative Controls (Cases 06, 17, 19, 20) | |
| Both stacks properly rejected all 4 impossible cases | |

### 20-Scenario Sequential Outcome Matrix (topology-mpp.json)

Green = Succeeded • Red = Failed • Gray = Negative Control (Expected Fail)

| | 01 | 02 | 03 | 04 | 05 | 06* | 07 | 08 | 09 | 10 | 11 | 12 | 13 | 14 | 15 | 16 | 17* | 18 | 19* | 20* |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Baseline | OK | FAIL | FAIL | FAIL | FAIL | CTRL | OK | FAIL | OK | FAIL | 5.1k | FAIL | FAIL | OK | FAIL | OK | CTRL | OK | CTRL | CTRL |
| AIR-GB | OK | MPP | MPP | MPP | FAIL | CTRL | FAIL | OK | OK | OK | 0 fee | OK | OK | OK | OK | OK | CTRL | FAIL | CTRL | CTRL |

Artifacts: wallet/native-data/benchmarks/{stock-02, ai-02, comparison-02/comparison.json}

LIVE DEMO & Q&A

# Live Web UI Demo (http://127.0.0.1:5173) & Key Takeaways

End-to-End Walkthrough on Alice (:8801) & Bob (:8802) with Local QVAC Copilot (:11434) • Open for Judge Q&A

TIMING SLOT
5:05 - 10:00



## Demo Part 1: Natural Language Policy

- Open Alice (:8801) in the Web UI (:5173).
- Enter natural-language routing constraint (fee cap, node exclusion, or privacy mode).
- Show QVAC extraction + deterministic compileConstraints() producing the SHA-256 paymentFingerprint & CompiledPolicy badge.

## Demo Part 2: First-Party RAG & Scoring

- Load node-scoped first-party history into SQLite + QVAC TurboVec (/v1/vector_stores).
- Run live Channel Scoring: inspect per-channel classifications (optimal, good, congested, avoid), 0-100 scores, Pareto weights, and retrieved RAG telemetry episodes.

## Demo Part 3: Dynamic Planner & RGB Send

- Run Dynamic Algorithm Selector: inspect EphemeralRoutingContext feasibility & choice (min_cost_flow MPP vs. multi_objective).
- Execute payment on Rust RLN daemon and observe automatic RAG episode ingestion.

✓ Ready for Judge Questions!

## Live Web UI Demo Walkthrough & Executive Scorecard

Interactive Demo on http://127.0.0.1:5173 (Alice :8801 & Bob :8802 + QVAC :11434) • Followed by Judge Q&A

LIVE WEB UI DEMO FLOW (2 MINUTES)

### Step 1: NL Constraint Compiler

`POST /api/constraints/interpret & /compile`

- Type natural-language routing rule in QVAC Copilot (e.g. fee cap, node exclusion, or single-path privacy).
- Inspect validated `CompiledPolicy` card and cryptographic `SHA-256` invoice fingerprint binding.

✓ Fails closed on ambiguous/fiat inputs

### Step 2: TurboVec RAG & Scoring

`POST /api/routing/score-channels`

- Load node-scoped first-party history (`20 / 100 / 500 / 2000 tx`) into SQLite and QVAC TurboVec vector store.
- Inspect live channel classifications (`optimal`, `good`, `congested`, `avoid`), 0-100 scores, and retrieved RAG hits.

✓ Bilateral physical liquidity guardrails

### Step 3: Planner & RGB MPP Send

`POST /api/routing/select-algorithm`

- EphemeralRoutingContext computes `singlePathPossible` & `mppPossible` in
- Dispatch to Rust RLN daemon with `multi_objective` Pareto weights.

✓ Auto-ingests outcome back into RAG

AIR-GB LIGHTNING — EXECUTIVE SUMMARY & Q&A SCORECARD

| REACHABLE SUCCESS RATE | MATCHED ROUTING FEES | NATIVE RUST RGB MPP | PLANNER & AI LATENCY |
|---|---|---|---|
| 81.25% | -78.95% | 3 / 3 | 0.66-1.35 ms |
| 13 / 16 Feasible (65% of All 20) | 1,200 msat vs. 5,700 msat | Multi-Path Splits & Returns | On-Node Rust Graph Solver |
| vs. 43.75% (7 / 16) in Baseline | ***Case #11 (Alice → Bob, 1 RGB):*** | 60+40 RGB & 40+40 RGB Splits | Qwen3.5-0.8B + EmbeddingGemma-300M |
| +85.7% More Completed Transfers | 0 msat (1 hop) vs. 5,100 msat (3 hops) | 190 RGB Returned to Alice (vs. 0) | 100% Local Loopback + 4 Guardrails |
| Same state hash: fc59bc78...8b45 | Saved by MPP return rebalancing | Fixed LDK multi-HTLC color_commitment | 52 / 52 Fixture & Safety Tests Passing |